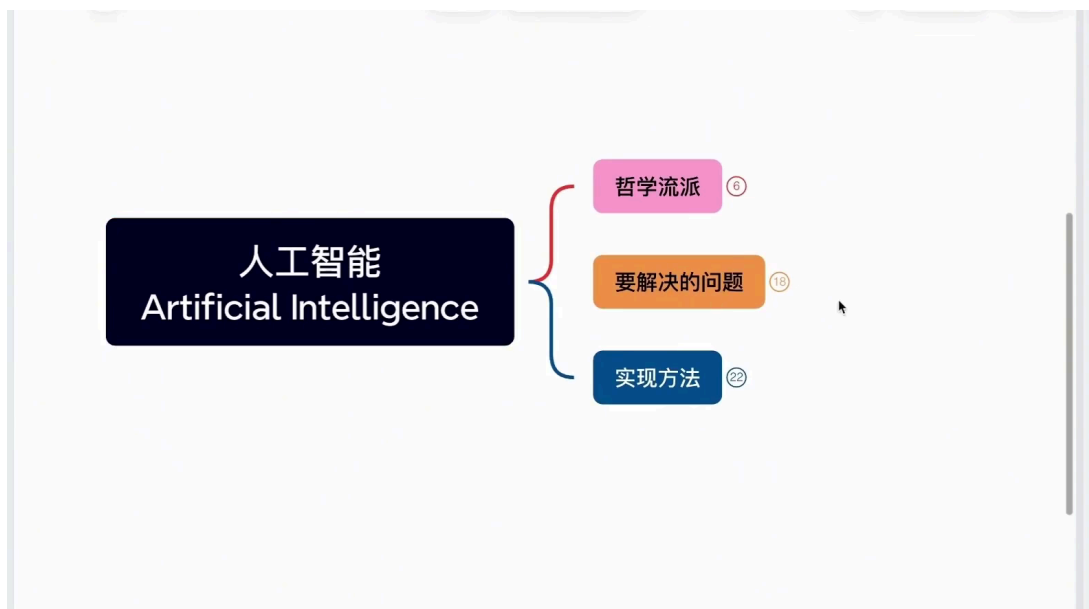


1.Agents底层逻辑之AI的分类 - 图文解说

一张关键画面对应一段完整解说，按原视频时间推进。

01. 先认识概念，才能无障碍交流



场景 1

完整内容

各式各样的博主在尝试给这一块下定义。没有意义，因为他们的定义不是广大人民群众的定义。广大人民群众说它智能，它就智能。

好，在正式开始我们的 Agents 内容之前，我们有必要花一点时间。我说的是“一点”，实际上有很多节课。我们需要花一些时间，把人工智能这个领域里的一些概念给同学们梳理清楚。

目的是什么？一方面是交流。以后同学们面试，可能会遇到面试官不停地给你蹦概念。其实这些概念你不懂，影响开发吗？完全不影响，一点关系都没有。后面做 Agents 开发，基本上这些东西忘完了也没关系。但是跟别人交流时，别人可能会说到一些概念，毕竟也可能遇到懂的人嘛。那个时候就容易懵，这个问题会存在。

第二个问题是，你不清楚这些概念，心里没底，总感觉挺害怕。人工智能领域的概念特别多，大部分跟平时说的 AI、Agents 开发没有关系。但你刷短视频，刷着刷着总感觉心里没底：怎么天天又公布一些新概念？其实这些概念本质上都挺简单，并不复杂，所以我们需要花时间认识它们。

除了交流，后面讲课时有时也没办法，需要蹦一些概念出来，你要知道我在说什么。这就是学这几节课的目的。我们不是要写学术论文，最重要的是理解清楚：它到底是什么东西，平时交流能够无障碍，目的就达到了。不是说我要写学术论文，因此每个概念都必须认识得非常清楚。有些概念跟数学密切相关，不可能在这里认识得那么清楚，所以我们以易懂为目标。

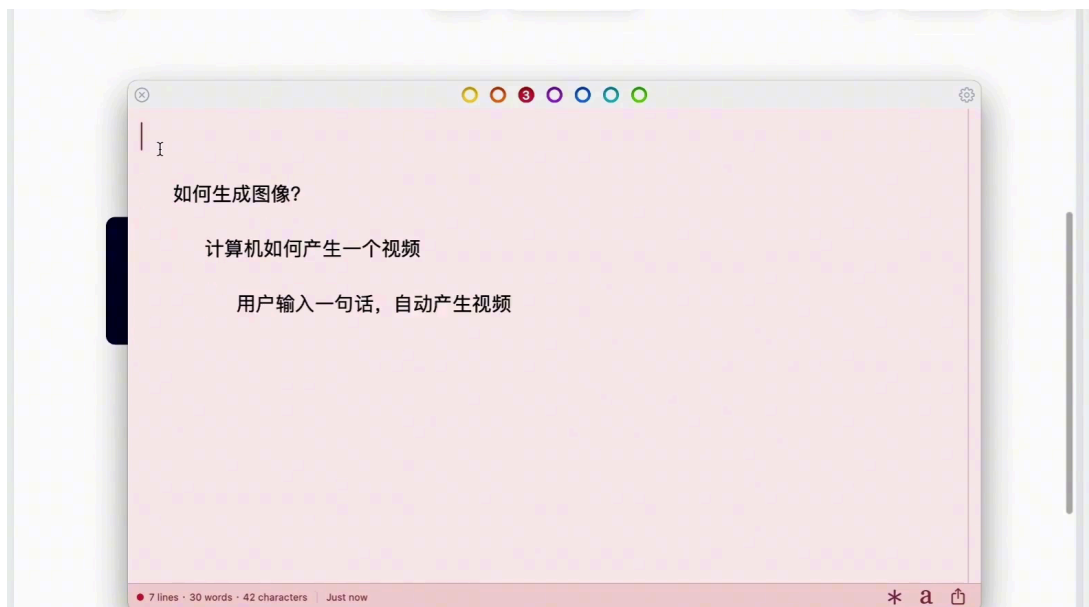
当然，按我这样解释，我跟你们讲的概念中，百分之九十九没有什么偏差。偶尔有一点点小概念的偏差，也不影响后续学习、工作和面试，放宽心就行。这需要讲几节课，一节肯定讲不完，因为概念太多。

首先，第一节是人工智能的各种分类以及实现方法。我们把这节课叫作“AI 的分类”，因为主要讲分类，这里会产生各种各样的概念。

画面说明

思维导图以“人工智能 / Artificial Intelligence”为中心，列出哲学流派、要解决的问题、实现方法三个入口。

02. 没有代码的思考为什么仍然重要



场景 2

完整内容

首先明确一下：这节课不包含任何代码。所以听上去会觉得有点抽象、有点虚，这是正常的。同学们都是工科思维。工科思维的典型特点是什么？一切要落地，你不落地，我就觉得虚。

程序员有句话：“Talk is cheap. Show me the code.” 你一定要会写代码，代码是落地、能运行的东西。你不写出代码，一切都是虚的，说了也白说。这是工科的思维。

其实这种思维不是一开始就有的。回忆一下，读小学、初中、高中的时候，哪里有落地，哪里有代码？全是做题，知道题该这么做，这是理科的思维。但现在离开学校很久，脱离了那种思维，就会觉得这节课比较虚。虚的东西不代表没有意义，它是有意义的。

我举个例子。假设这个世界没有 AI，连“人工智能”这个词都不存在，一切归零。我们进入一个平行宇宙，那里有计算机、有开发，但是没有 AI。老板让你实现一个功能：用户输入一句话，自动产生一个视频，也就是文生视频。你脑袋是不是懵的？怎么办，是不是就辞职了？没法干这个事情嘛。

但假设你必须做，不能辞职。你不做，整个宇宙就会爆炸；为了拯救全宇宙，必须完成。我们做个想象。你想想怎么做？首先完全下不了手，于是你会问：下不了手是因为什么？

第一个问题：计算机怎么自动产生视频？这还在工科的角度，都还没到理科。你先想“计算机如何产生一个视频”，把它想通，才觉得能实现后面的需求。

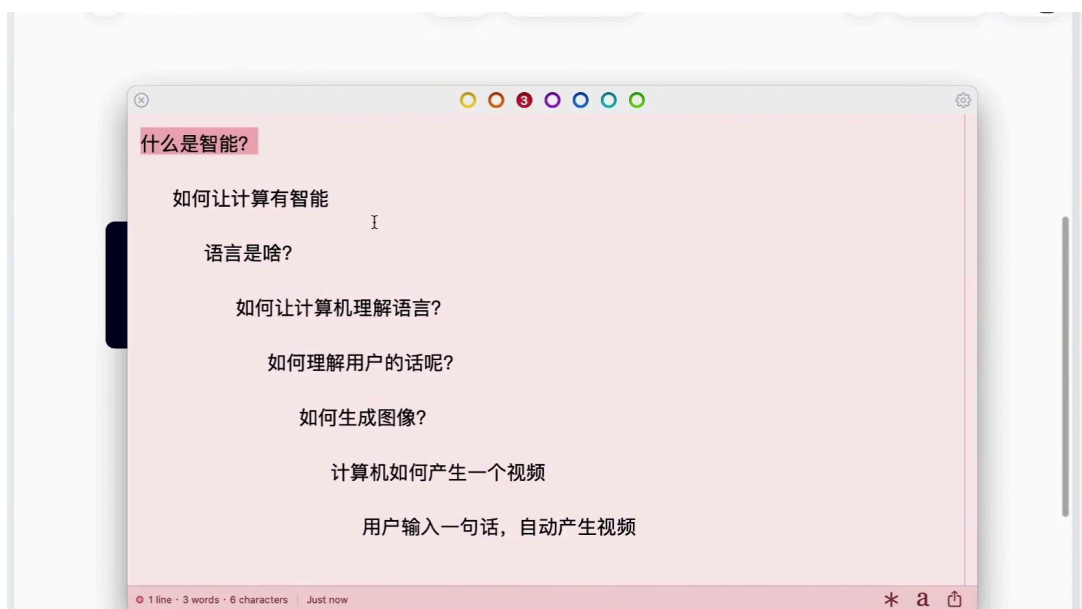
假设查了一些资料，发现视频原来就是一帧一帧的画面，无非是生成画面，再把画面组装起来，不就是视频吗？是不是有方法了？请问这时写代码了吗？没有。但是这个思考过程重要吗？重要。你不思考，怎么自动产生视频？我都还没说根据用户的话产生视频，只是“产生视频”本身，你都不知道怎么做，得先把方法搞通。

想清楚后，又遇到下一个问题：视频由一帧帧图像组成，那如何生成图像？图像还不能乱生成，要根据用户的输入生成。要搞定它，是不是先要搞定一个前置问题——如何理解用户的话？而要理解用户的话，又会遇到更大的问题。

画面说明

便笺从“用户输入一句话，自动产生视频”，往上列出“计算机如何产生一个视频”和“如何生成图像”。这是讲解中从需求向前置问题追问的过程。

03. 从文生视频逐层追问到智能



场景 3

完整内容

更大的问题是：如何理解语言本身，如何让计算机理解语言？要搞定这个问题，又要思考“语言是什么”。你会发现语言里包含智能。要让计算机理解语言，是不是先要让它有智能？如何让计算机有智能？

你看，除了最下面的实现层面，上面的哪个层面有代码？全是思想、方法、模式。继续追问“什么是智能”，恭喜你，进入哲学领域了。这个只有哲学能回答，甚至哲学都回答不出来，只能讨论：什么叫智能、什么叫人、什么叫我。到这个领域，我们就没法讨论了。

但是要落地一个具体需求，当这个世界一片虚无，没有前人研究过这些东西时，你只能从理论出发，一个理论一个理论去趟、去搞定。搞定之后你会发现，不仅能做文生视频，还能做其他事情，因为这是更本质、更高维度的问题。

所以这节课的抽象不是没有意义。只是在工科生的思维中，它没有落成代码，看起来没意义；在理科生的思维中，这些却是必须解决的问题。最顶层的问题哪里有代码？没有，甚至“计算机”都可以换成“机器”，不一定是计算机，也可以是其他东西。只有思想。

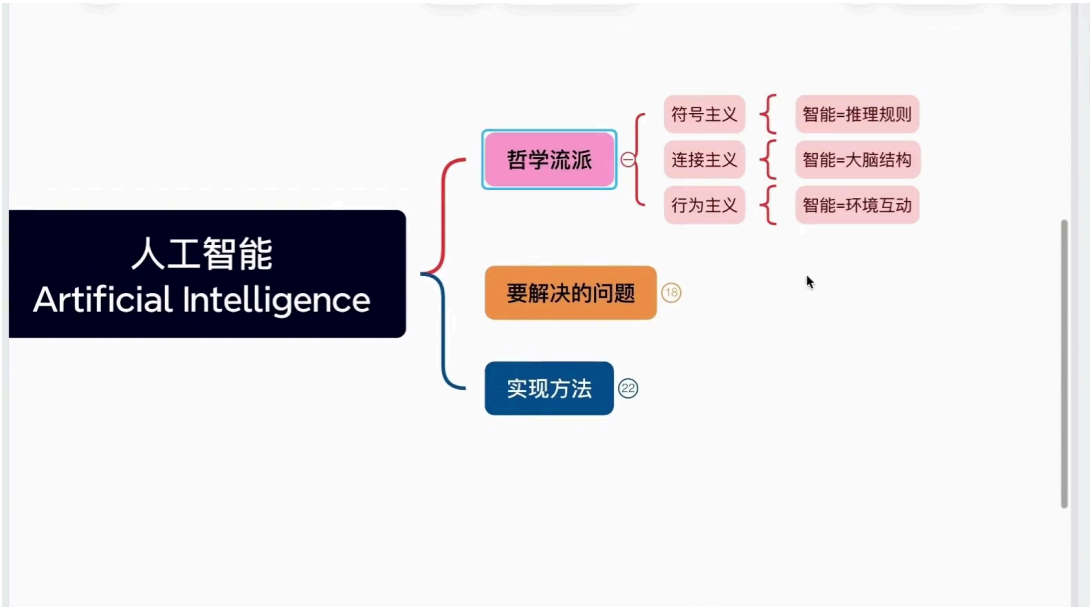
这种情况在学术界很常见。探讨这些学术问题时，要习惯没有代码，因为它们本身就不产生代码，而是思想。思想有层级：要先搞清这个问题，才能搞清下一个，再搞清下一个。每一层也没有代码，只是更具体的思想。思想有不同的抽象级别，一层一层抽象上去。

基于这样的理解，再来看人工智能。第一个要回答的问题是：什么叫人工智能？学术界有一些定义，有兴趣可以问 AI 了解那段历史；也没有必要一定去问，它不影响我们的交流。我告诉你，人工智能到现在没有一个准确的定义，没有人能把它到底是什么完全说清楚。

画面说明

便笺完整列出：用户输入一句话自动产生视频 → 计算机如何产生视频 → 如何生成图像 → 如何理解用户的话 → 如何让计算机理解语言 → 语言是什么 → 如何让计算机有智能 → 什么是智能。上方问题具有更高的抽象层次。

04. 如何理解人工智能与智能的共识



场景 4

完整内容

没有人能把人工智能到底是什么完全说清楚。“人工”好理解，人造出来的东西嘛。那什么叫“智能”？怎么理解？你回答不了，这是哲学家的问题。哲学家有清晰答案吗？没有。但是有共识。

什么叫共识？比如现在的文生视频，跟 AI 聊天，表现得很像人。最近 GPT 又出来一个新版本，叫 GPT Live〔此名称按原画面字幕保留〕，同学们有没有用过？跟它像打电话一样直接对话，它会接话、会想，也会“嗯”“啊”，表现得特别像人。大家认为它挺智能，就有共识了。它又是人工造出来的，那就是人工智能。这里纯靠共识。

再回到十几二十年前，一些游戏里的 NPC，特别是一些 3A 游戏，NPC 挺厉害：每天自己起床、工作、吃饭、睡觉，还可能跟别人打架，表现得像一个人。像不像智能？也挺智能。大家有共识，叫它人工智能，可以。现在则觉得稀松平常，没什么智能；研究清楚算法后，发现也没什么，就不把它叫人工智能了。

自动泊车算不算人工智能？这就有争议，不好说。给一个包含十万个数字的数组排序，挺厉害，放在很早的过去，大家可能觉得智能。现在不是稀松平常的事情吗？那算什么智能？没有人能给一个清晰定义。

网上各式各样的博主尝试下定义，没有意义，因为他们的定义不是广大人民群众的定义。广大人民群众说它智能，它就智能。这一块完全靠共识，不用纠结到底什么是人工智能。大家觉得智能就智能了，好吧。

人工智能有三大哲学流派。它主要讨论：我要实现机器的智能，该沿哪个方向走？它自己不去落地，只探讨“怎么让机器有智能”。这是个首要问题。智能是什么回答不清楚，那先模糊着，但要想想怎样让机器有智能。

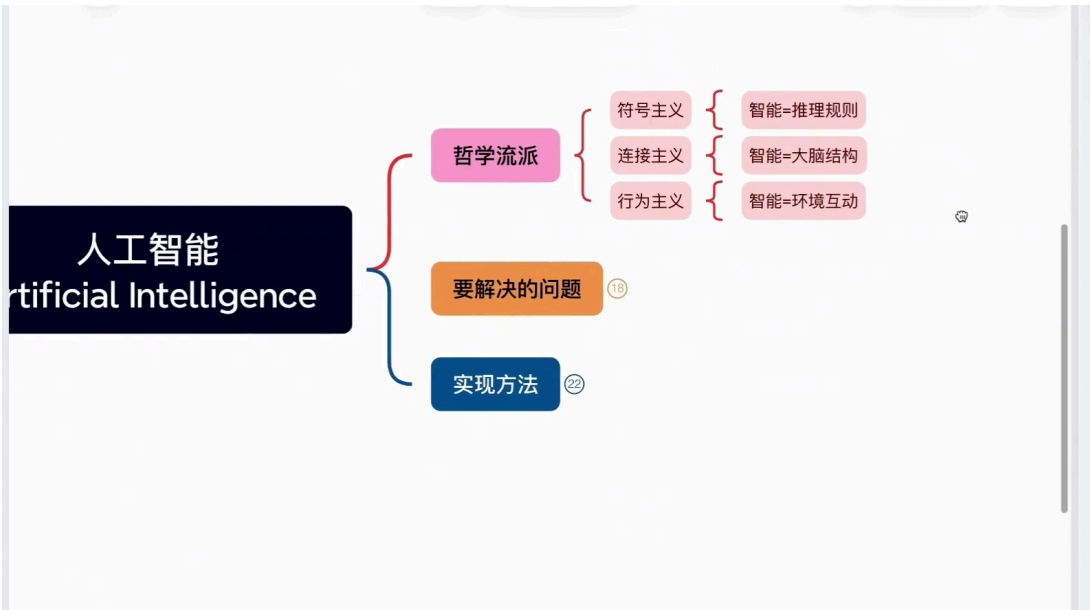
于是诞生了三大学术流派，是纯学术讨论，没有落地的东西。但没有这些讨论就没法起步：根源问题都没解决，怎么实现智能？连方向都没有，大家脑袋都一片懵。上个世纪五六十年代——准确来说，应该是四五十年代——就已经在讨论这件事。当时刚有计算机不久，就讨论人工智能，很有先见之明，但当时肯定落不了地。

这一块作为知识了解即可，跟平时交流关系也不大。首先，符号主义认为智能等于推理规则。人类在世界上生活，学习了很多规则，特别是逻辑规则。

画面说明

导图展开三大哲学流派：符号主义对应“智能=推理规则”，连接主义对应“智能=大脑结构”，行为主义对应“智能=环境互动”。口述先用聊天、NPC、自动泊车和排序举例，再引出这些方向。

05. 符号主义、连接主义与行为主义



场景 5

完整内容

比如“人固有一死，苏格拉底是人，所以苏格拉底一定会死”，这就是逻辑推演。符号主义认为，人类在社会中体验到各种逻辑，把这些逻辑告诉计算机，它不就有智能了吗？它沿这个方向走。

符号主义曾经风靡一时，现在很多地方也沿用。例如游戏 NPC，固定告诉它一些规则、算法，让它照着走。以前的人工智能比赛里，一个小车找迷宫，不是遥控器控制，而是自己找路走出迷宫：通过规则、算法、逻辑，让机器沿着逻辑走，最终找到出口。这些是符号主义的延伸应用。

大概了解它的哲学思想就行。它其实没说多少东西；按这里的讲法，并没有一本书专门来介绍“符号主义”这个思想。它是哲学流派，认为沿这个方向可以实现人工智能。

连接主义不是这样看。它认为世界规则太复杂，人的推理逻辑都不一定理解正确。要实现智能，就得把机器造成跟人一样。人靠什么有智能？靠大脑。那就把机器弄成大脑的结构，大脑中有很多神经元连接，研究神经元是怎么回事。

当时提出连接主义的人中，有很多研究人类神经系统的神经学家。我们也在机器里模拟大脑结构，不就有智能了吗？让它自己去理解，这是连接主义的思想。现在肯定是连接主义的时代了。

行为主义则说：光有一个大脑在那里有什么用？得跟环境互动。人的大脑确实复杂，那蚂蚁呢？我记得蚂蚁好像没有大脑皮层，但它也有智能，分工协作多么智能。它靠探索环境，环境给反馈，慢慢学到东西。这是行为主义的思想。

有人问，行为主义现实中怎么落地？它不考虑落地，这是早期哲学流派在探讨。上世纪五六十年代那么早，哪里有落地，只是在讨论怎么让机器有智能。作为了解就可以，平时这些词用得少，听到不要惊讶，知道是历史上的哲学讨论即可。

那现在的 ChatGPT 等 AI 属于哪套流派？我不知道。实际上都有。比如 Transformer 这套架构，属于哪套？也都有。符号主义、连接主义并不互斥，不是非你即我，可以交融。中间某些环节有推理规则，也可以模拟大脑结构，最后还可以通过反馈重新学习。

现在也没人在意某个功能用了哪个主义。假设将来小张当了 AI 训练师或算法工程师，要训练 AI、做人工智能产品。有一天领导问：“小张，现在这套产品属于哪套哲学流派，是哪个主义？”符号主义、连接主义、行为主义都不属于，我属于 KPI 主义！KPI 让我做什么，我就做什么，哪里拿钱多，我就做什么。

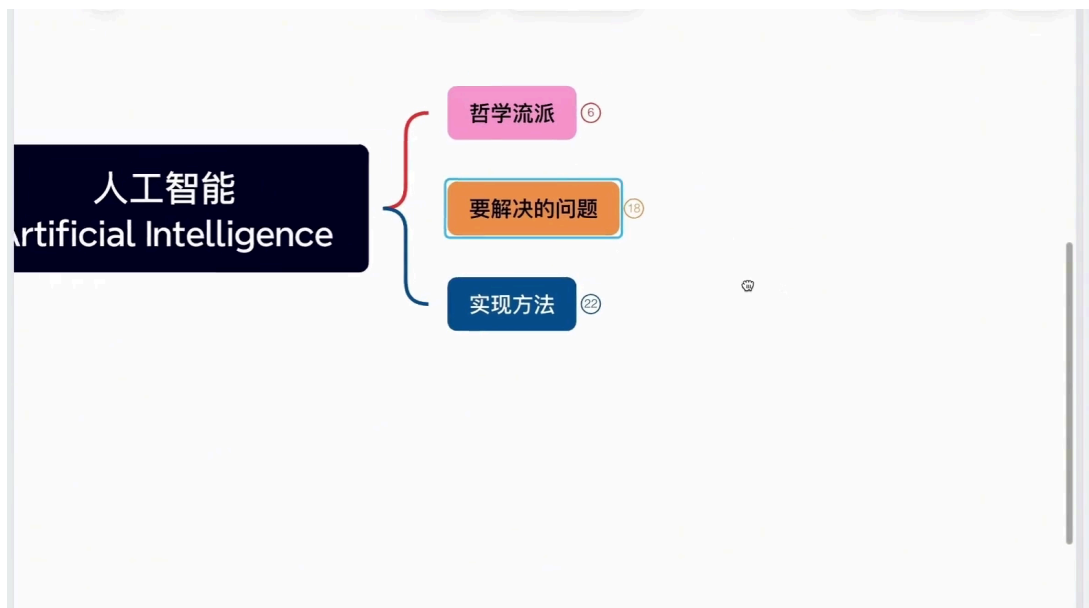
没有人这么讨论产品。但我们要知道，历史上有这些哲学讨论。硬要分，也可以看某个模型训练中的环节是哪种思想的延伸，只是这件事没什么意义，没人讨论。纯学术的东西，不用纠结。符号主义、连接主义、行为主义，我讲清楚了吧？

人工智能有很多划分方式。这里列举哲学流派和要解决的问题两个角度。从别的角度还可以有不同划分，只是一些术语我们平时实在用不到。

画面说明

画面保留三条并列关系：符号主义—推理规则；连接主义—大脑结构；行为主义—环境互动。它们在图中作为哲学流派并列，不是产品功能列表。

06. 分类角度可以不同，不能互相硬套



场景 6

完整内容

分类方式还可以有很多，只是有些术语平时用不到。我们这门课的目的，不就是先知道并理解常见术语吗？那些不常见、根本用不到的，就不扯了。

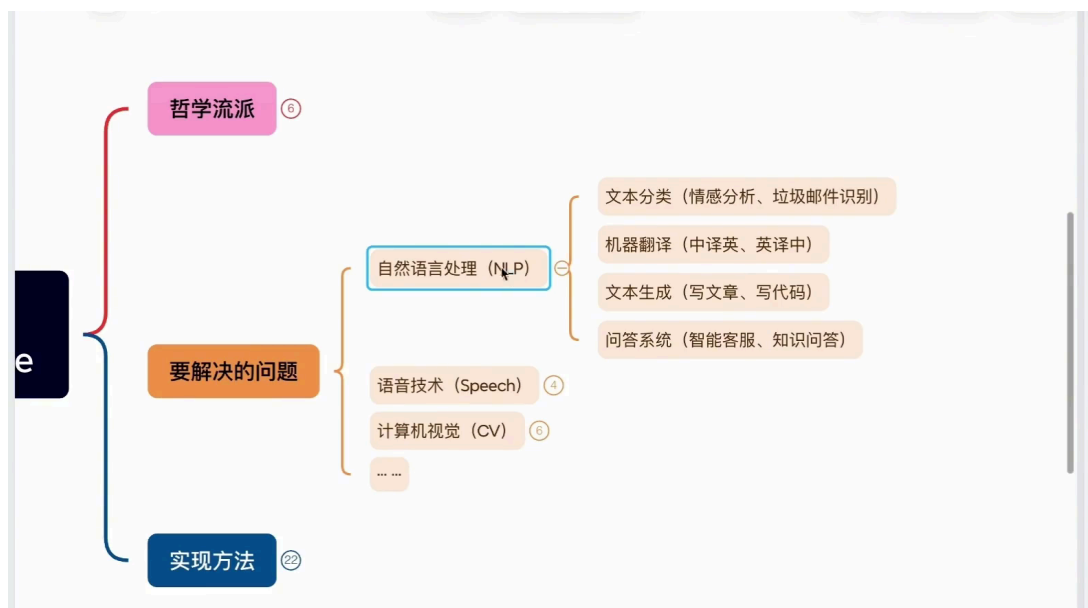
比如给人分类，也有很多方式。按性别分，有男人、女人；按年龄分，有成年人、未成年人。那你说未成年人是什么性别？都有可能，对吧？这是不同角度。按道德可以分好人、坏人；按职业就更多，有前端工程师、后端工程师、Agents 产品开发工程师。分类方式很多。

这里介绍两个主流角度：一个是哲学流派，一个是要解决的问题。我不管你用什么东西做，先看你要解决什么问题。按这个角度，有几个比较常见的术语。其实问题还有很多，比如科学领域的天气预测、蛋白质分子结构、医学领域的问题，也都有各自术语，只是我们平时少见。所以先看这三个，其中一个跟语言处理相关的。

画面说明

画面切换到“要解决的问题”分支，另两个入口仍是哲学流派和实现方法。口述通过性别、年龄、道德与职业说明不同划分角度。

07. 自然语言处理研究什么问题



场景 7

完整内容

语言处理相关的问题，就是让计算机拥有理解语言的能力。它管怎么做吗？不管。可以探讨很多种做法，总之把事情搞定就行。

比如文本分类，我要识别垃圾邮件，哪些是垃圾，哪些不是，要分类。怎么识别？得理解语言，不理解语言很难分。机器翻译就是中译英、英译中，或者其他语言间的翻译。文本生成就是写文章、写代码。

那现在的 Transformer 属于这里哪一类？哪个都不属于，或者说哪个都属于，它们就不是一种分类方式。这些是要解决的问题。你可以用 Transformer 解决这个问题，也可以解决另一个。比如用它做文本分类模型，也可以做问答系统，像 ChatGPT 那样，都可以。

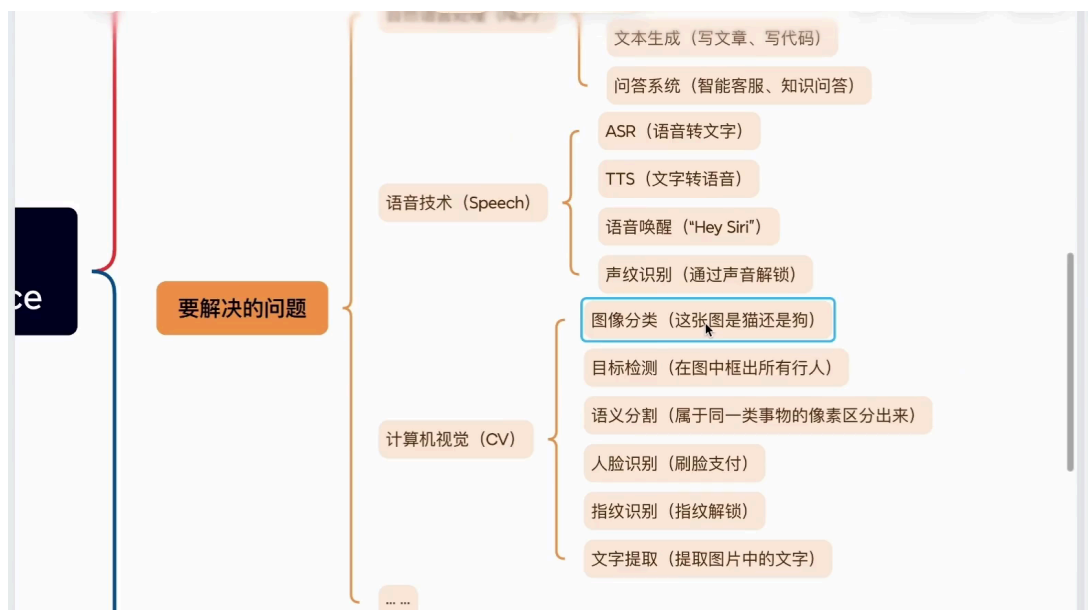
这里不管具体手段，只管要解决这个领域的问题。自然语言处理，是不是要理解语言？语言到底是什么，单词之间存在什么关系？它研究的是这个。不是规定用哪一种 AI 能力实现，用什么能力都可以。它主要研究语言本身的特点，再结合各种 AI 技术，看能不能实现。

说白了，你可以把它理解为应用场景、一个 APP。做这个 APP，就要研究其中的特性。语音技术也一样，有语音转文字、文字转语音。一些术语会经常遇到：TTS 是文字转语音，ASR 是语音转文字，都属于语音相关技术，还包括语音唤醒。

画面说明

自然语言处理（NLP）下有文本分类（情感分析、垃圾邮件识别）、机器翻译（中译英、英译中）、文本生成（写文章、写代码）及问答系统（智能客服、知识问答）。这些是问题类别。

08. 语音技术、Transformer与应用场景



场景 8

完整内容

比如“Hey Siri”，还有声纹识别，通过声音识别或解锁，这些属于语音技术。它不管 AI 怎么实现，这个领域主要研究语音的特点：频率、音色、音调，怎么数字化；数字化之后，再研究用这套 AI 方法或那套方法来实现。

有些同学有个误区：现在 GPT 很火，GPT 属于 Transformer 架构，对 Transformer 理解不够，就错误地以为它是“做语言的”。不是，它也能做语音技术。现在做语音、做语言，很多都是用这个架构。只是 GPT 把它做成了语言类的东西。文生视频、文生图、语音合成都可以用它。按这段课程的讲法，目前它最流行，基本都在用它。它到底是什么，我们后面慢慢听就知道了。

这些是具体业务场景里要解决的问题。自然语言处理解决了，就能理解你说的话，可以写文章、写文案，甚至写代码。代码也比较贴近自然语言，这些都是研究范畴。

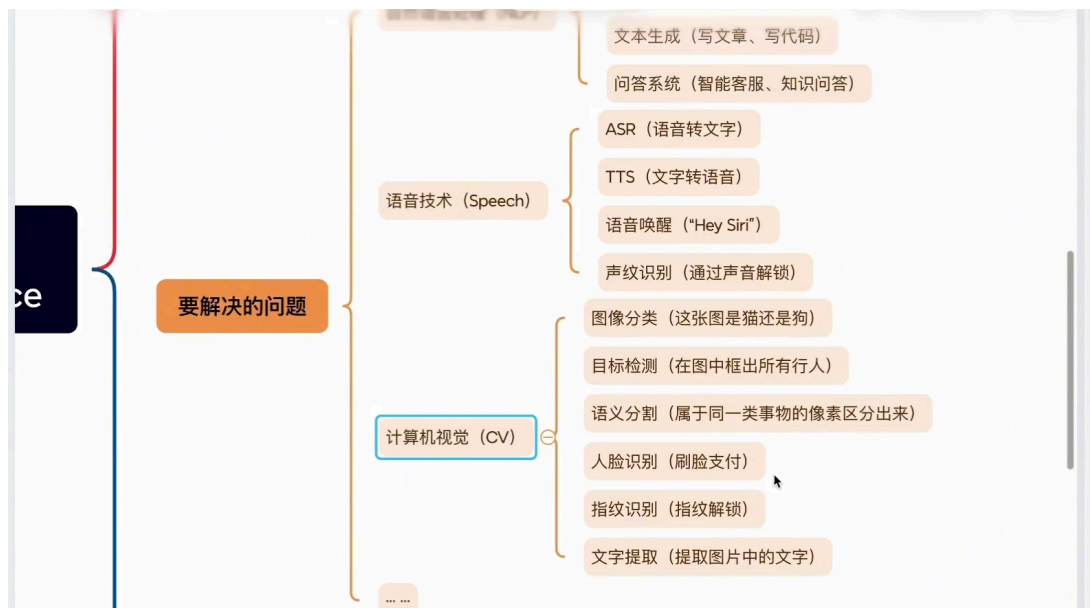
语音有它自己的范畴和特点，语音和文本肯定不能完全划等号。要尊重每一块业务领域。这是领域要解决的问题，不是某一个模型。你要说里面有技术，那也是为了解决这些问题。比如文本怎么切分成 Token，后面会讲；语音怎么切分、怎么转换成 Token，也有各自要研究的问题。

这跟我们学的 Transformer 模型不是一个层面的事情。还有计算机视觉，比如图像分类：这张图是猫还是狗？目标检测：通过摄像头知道商场今天的客流，来了多少人，走了多少人，老人多少，不同年龄段多少。

画面说明

语音技术 (Speech) 分支列出 ASR (语音转文字)、TTS (文字转语音)、语音唤醒 (“Hey Siri”) 和声纹识别 (通过声音解锁)；下方可见计算机视觉分支。

09. 计算机视觉的常见任务



场景 9

完整内容

通过图像识别，有可能把不同年龄段分出来，但这个分类不是特别准确。

还有语义分割。比如 B 站有“避免弹幕遮挡人物”的功能，要把人物对应的像素点全部框出来：这个区域是人物，是同一类东西。把它框出来，才能抠出来，这是语义分割要做的事情。

人脸识别、指纹识别、文字提取——也就是提取图片里的文字——这些都是计算机视觉领域研究的问题。不是规定一个 AI 模型，只要能解决问题就行，管你用什么模型。每个问题都有其场景特点。

比如你入职一家做 Agents 相关业务的公司，它说“我们公司做 NLP”，不是说 NLP 是一个模型，而是说公司做的是这个领域，是语言方面的事情。

画面说明

计算机视觉 (CV) 分支列出图像分类 (猫或狗)、目标检测 (框出行人)、语义分割 (区分属于同类事物的像素区)、人脸识别 (刷脸支付)、指纹识别 (指纹解锁) 和文字提取 (提取图片文字)。

10. 用 Web API 类比业务领域与实现技术



场景 10

完整内容

进一步细化，公司可能做文本生成、文本分类、智能客服，或者专门研究 NLP 各子类别中的特性。用什么技术实现？什么好用就用什么。

就好比我要开发一个 Web 应用，暴露一些 API。能不能用 Java？能不能用 Go、Python、Node.js？都可以。它不规定必须用什么，哪个技术好用就用哪个。

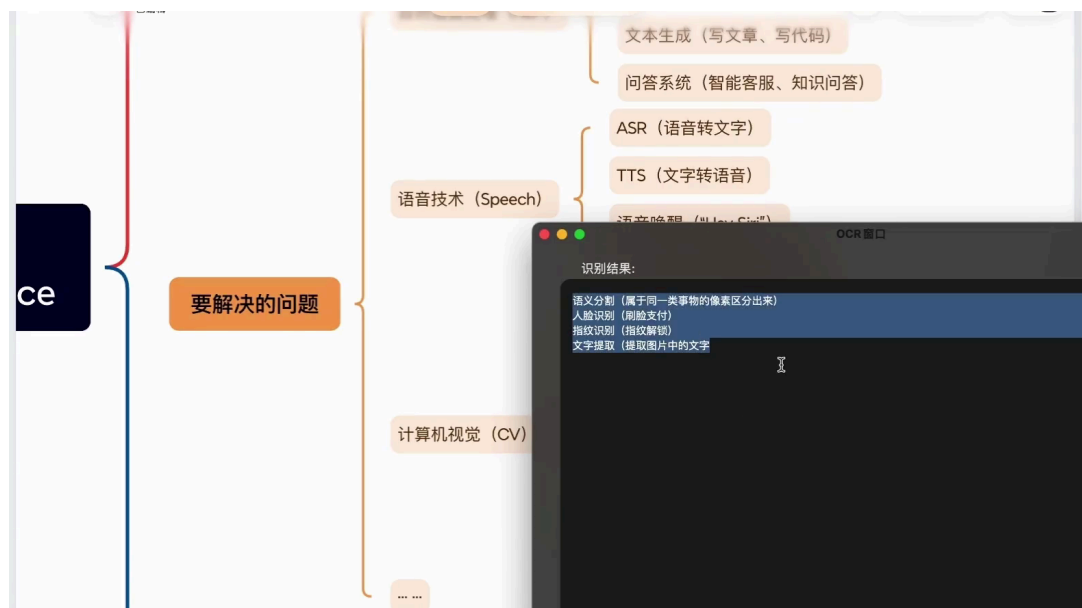
比如一些小公司，特别是国外公司，会用 Go，看重性能；或者 Java 生态成熟，就用 Java；一些 AI 工程用 Python；如果团队全是前端，那就用 Node.js，反正前端都会。没有限定一定用哪项技术。

但不管用什么开发 Web 应用，有没有共性问题？数据库是不是问题，怎么存储、访问，一般会通过 ORM 来做。有没有三层架构的问题，有没有 AOP、依赖注入这些事情？各式各样的 Web API 应用都有这些特性。

这些领域研究的是自己的特性，而不是只研究某一种具体实现。现在流行用 Transformer，但在不同历史时期，实现方式多得很。比如文字提取，以前常见的就是 OCR。你看，我这个截图工具都有 OCR，也就是文字识别，用它来识别文字。它跟目前的 Transformer 没有直接关系，实现方式有很多。

画面说明

便笺写有“web 应用 / API”与 Java、Go、Python、NodeJS。四种语言对应同一种应用需求，用来说明应用领域不限定实现技术。



场景 11

完整内容

这个图像能不能用 Transformer 那种模型识别？也可以。把数据截图下来，问一个模型〔此处具体名称识别不清〕，它也能识别出来，没问题。实现方式有很多。这是业务场景，每个领域有它要解决的问题。

我讲得很细，就是怕大家模糊。为什么不能模糊？因为这些术语靠背是背不住的，只能理解，知道原来是这么回事。

比如 TTS，现在按这节课的讲法，基本靠 Transformer 模型合成语音。但它是唯一的实现途径吗？不是，也有别的途径。只是当下 Transformer 打赢了，效果确实好，所以大家用。它一定不是唯一的。

哪怕有一天 Transformer 不流行，这个业务场景仍然在，会有别的实现手段，但场景里的特性还在。这个角度就是“要解决的问题”。

最后才是“实现方法”。你说了那么多，怎么落地？前面都在学术交流，这里其实也还是学术交流，只是更贴近落地。实现成代码了吗？也没有，全是论文。前面提了那么多问题，我该怎么落地，就要开始想这个问题。

画面说明

截图工具的 OCR 窗口显示识别出的中文条目，包括语义分割、人脸识别、指纹识别、文字提取；背景仍是业务问题导图。这是讲者现场演示文字提取的结果。

12. 实现智能的方法：知识工程、演化计算、机器学习



场景 12

完整内容

这里也要讨论，有很多种落地方式，比如知识工程、演化计算。

知识工程，就是把人类已有的知识总结起来，形成一种数据结构。什么数据结构？这里不管。可能是一个知识树，让计算机探索这棵树，学到东西，产生智能，解决各行业的问题。每个行业有自己的知识树，大概这样理解。

演化计算有点像达尔文的演化论：一开始肯定做不好，做不好的淘汰，做得好的留下，好的跟好的再生下一代，是这样一套理论。虽然是理论，但里面有数学演算、数学证明，不是纯粹遐想，背后有数学原理。它仍不管代码怎么落地，只认为这条路可行。

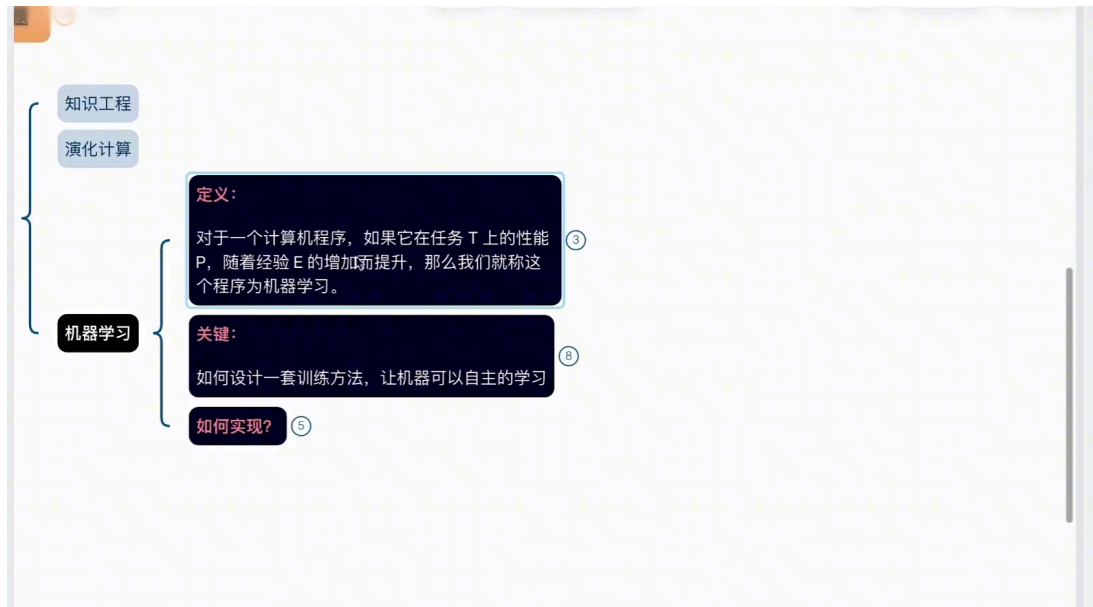
按这里的讲述，这些已是过去的路线，现在实现智能、解决各行业问题，主要靠机器学习。所以这门课讲概念，主要讲机器学习里的相关概念。机器学习是什么？看清三块也就够了。

再说一次，机器学习本身写代码吗？也不写，也是在讨论。还记得刚才的例子吗？要先想清很多东西，才能最终写代码。

画面说明

“实现方法”分支展开知识工程、演化计算和机器学习，其中机器学习使用黑色节点突出。

13. 机器学习的定义



场景 13

完整内容

先要想清很多东西，才能写代码。高层次的事情都没想清，代码写不了。

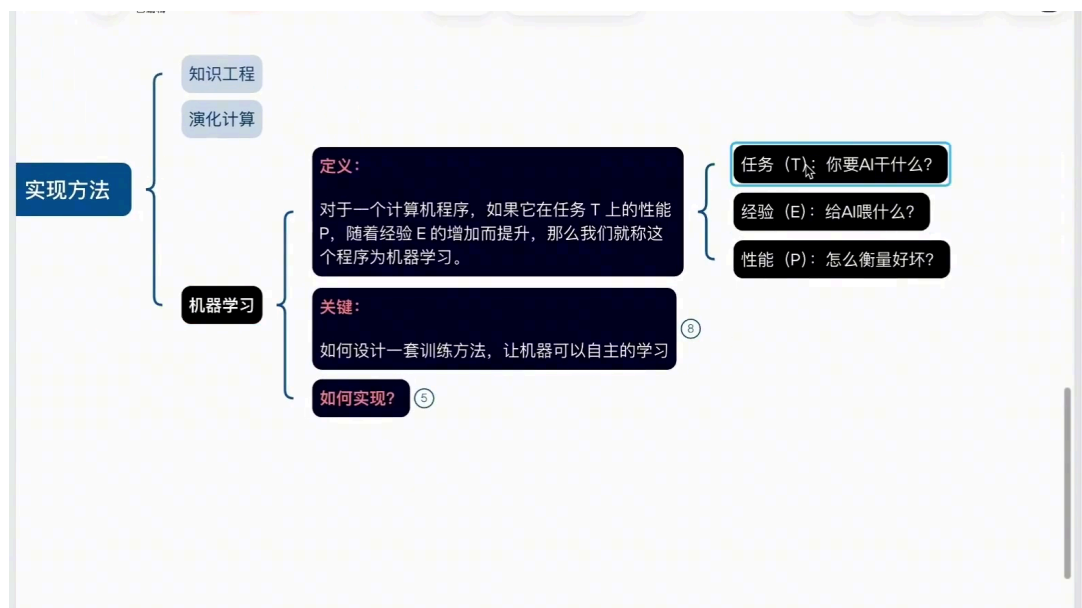
机器学习要思考：怎么才能让机器智能？它提出这样一个定义：对于一个计算机程序，如果它在任务 T 上的性能 P ，随着经验 E 的增加而提升，那么我们就称这个程序为机器学习。

说的是什么？看三个维度。第一个，任务是什么？

画面说明

导图定义为：对于一个计算机程序，如果它在任务 T 上的性能 P ，随着经验 E 的增加而提升，那么我们就称这个程序为机器学习。下方指出关键是设计训练方法，让机器自主学习。

14. 任务、经验和性能，以及跨领域的智能



场景 14

完整内容

任务可以是任何任务，随便什么任务。不是说机器学习只解决 NLP，它不管这个。角度都不一样，NLP 有 NLP 的问题，Speech 有 Speech 的问题，CV 有 CV 的问题，各有各的问题。要解决这些问题都需要智能，但还必须搭配具体领域中的东西。

比如 Java 只能写 Web 应用吗？不是，控制台应用、桌面应用也可以写。Java 是通用型语言，我们把它叫图灵完备。在这里直观地说，就是计算机能做的事，它都能做，只是通常用来写 Web 应用。

所以机器学习不是为解决某一个领域，而是解决各领域里“智能”这个点。比如语音技术有自己的特点：怎么把语音变成数字化的东西，怎么数字化、压缩、分割，这是语音技术的问题。但是不管怎么做，最后都遇到同一个问题：智能怎么办？实现方法这一块探讨的是怎么实现智能。

把智能的问题探讨清楚，各行各业是不是都可以利用这个智能，结合行业特点实现业务需求？能理解吗？我已经讲得很细了，网上许多教程不会把这些抽象概念讲得这么细，听完可能一头雾水。我尽量让大家认识清楚概念的意思。

机器学习定义的三个关键词，先看任务：你要做什么。比如识别一篇文章，或者写一篇文章，这就是任务。

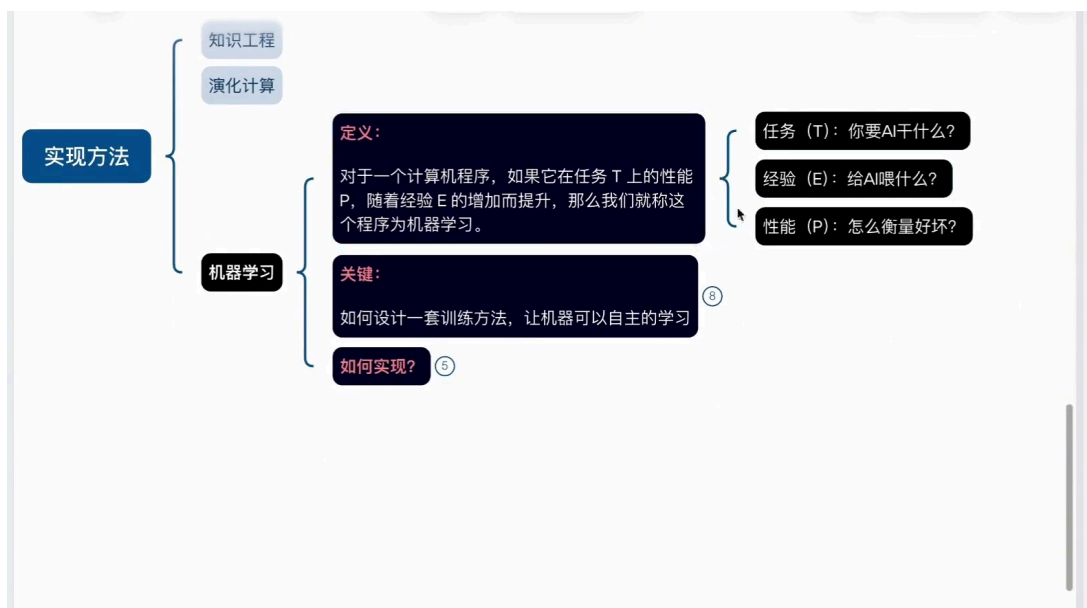
经验 E，代表给 AI 喂什么。要写文章，得见过很多很多文章。所以机器学习有一个特点：要进行大量数据的训练。这里不管具体怎么训练，只是在提出这件事。但它是不是又把事情往前推进一步，具象化了一步？具体落地就是一步一步，从抽象慢慢具象，到最后可以落地，落地就自然而然了。

从 AI 诞生到 Transformer 模型，按照这里的讲法，是接近一百年的探讨过程。其中也走过很多其他路线。那些历史路线完全没用吗？历史总会轮回，说不定哪天又翻出来用。比如连接主义，过去曾被打入死牢，没人愿意探讨，只是提出来就被认为是天方夜谭。

画面说明

定义右边展开任务 (T)：你要 AI 干什么；经验 (E)：给 AI 喂什么；性能 (P)：怎么衡量好坏。它们与左侧定义一一对应。

15. 用写文章与下棋理解 T、E、P



场景 15

完整内容

连接主义提出后，曾被认为是天方夜谭。有些人甚至从数学角度证明它根本不可行。我告诉你，那个数学证明至今仍然正确，不过后来的情况跟证明的前提出了一点差异。想了解这段历史，可以自行了解。以前是符号主义的天下，在大学研究连接主义，根本拿不到经费。大家觉得只是烧钱，一群人在那里讨论，可以这么理解。

这里不管怎么落地，只说要把数据喂给机器，怎么喂并不管。像人一样，要见过很多文章，才能写出好文章。

性能 P 关注什么？怎么衡量好坏，怎么评判文章写得好不好？比如由人类打分，就可以衡量。把这个事情说清楚即可。后续怎么写代码、怎么训练，它不管。你只要在做这些事，都在机器学习的范畴里。

所以很多大学 AI 课程都讲机器学习，因为现在流行。我给它加黑，是因为这是目前最流行的实现手段，在这个大前提下学习。具体怎么学习，还有很多路线，但大范畴都叫机器学习。

做的事情就是不停地给它喂数据，它能学会做这件事，再通过一种方法衡量做得好不好。不好就回去重来，加大训练量，再调整、再学习。具体怎么做，这个定义不管；只要在做这样的事，就算机器学习。

比如下棋，AlphaGo 是一个典型成果。任务是什么？下棋。喂什么数据？无数棋盘数据、棋局数据，不光著名棋局，大大小小的棋局、人类历史上的棋谱都喂给它，让它学。

怎样衡量好坏？看结果，赢了还是输了，赢多少、输多少，围棋就通过这些衡量。随着经验 E 增加，喂的东西越来越多，这件事能做得越来越好，并且有一套标准衡量好坏，就符合机器学习的定义。

有人说，这不是很自然吗，看得越多肯定越好。那可不一定，它不是人，程序可能看得越多反而越傻。就像书呆子，学得越多，人越傻，因为不能融会贯通，知识没有内涵。每个知识对它都只是冷冰冰的文字，像背地理、政治、历史，一个个人名彼此没有关联。这样学得越多，人可能越傻。

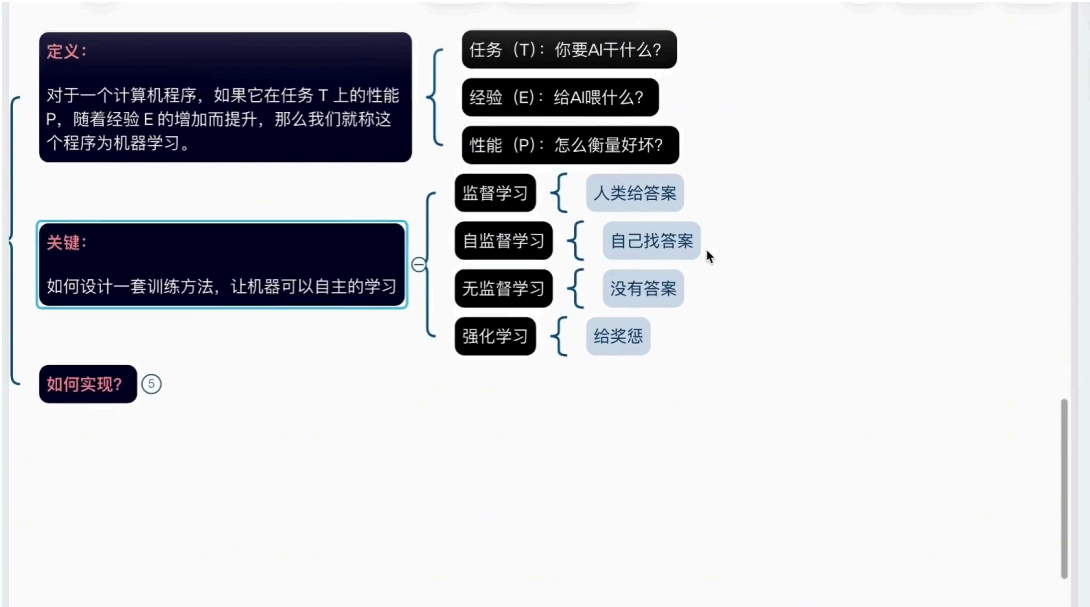
这个定义不负责落地，不用在这里想怎么落地；只要在这个范畴内，就属于机器学习。要把机器学习摊开讲，是一门非常庞大的课程，因为做法太多。

那么关键点是什么？如何设计一套训练方法，让机器能够自主学习。机器学习的核心就是“要学”。我喂数据的目的是什么？你要学。连学习能力都没有，我给你喂数据有什么意义？

画面说明

画面保留 T、E、P 的三个问题与机器学习定义。讲解以文章、棋谱和输赢结果逐一对应任务、经验和性能。

16. 关键是设计能够学习的训练方法



场景 16

完整内容

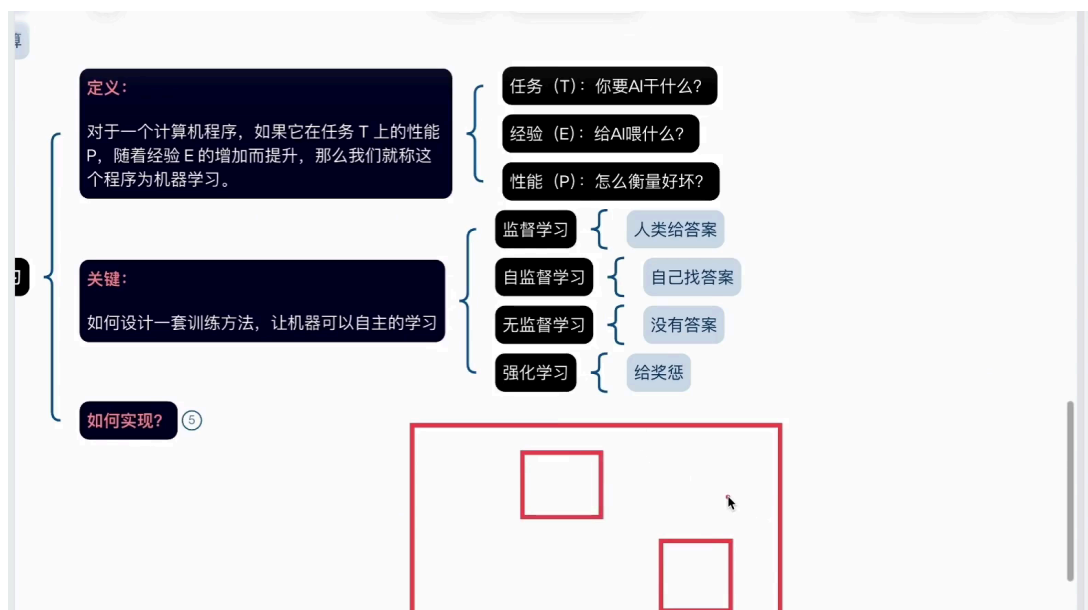
你没有学习能力，喂数据就没有意义。在控制台打印一百遍棋谱，有什么意义？没有。机器得学，所以关键是如何设计训练方法，让它能够学习。

这里又开始讨论了，还是不落地，但又往前推一步，提出方法论。我们把它叫“学习范式”或者“训练范式”。落地了吗？没有，一行代码都没有，还在讨论。记得前面的追问链吧，要一直讨论到最后一步，才可以落地，中间还有很多问题没解决。

画面说明

关键节点右边展开四类：监督学习—人类给答案；自监督学习—自己找答案；无监督学习—没有答案；强化学习—给奖惩。

17. 监督学习与数据标注



场景 17

完整内容

单看这几种学习类型，非常好理解。第一种叫监督学习，就是人类给答案。

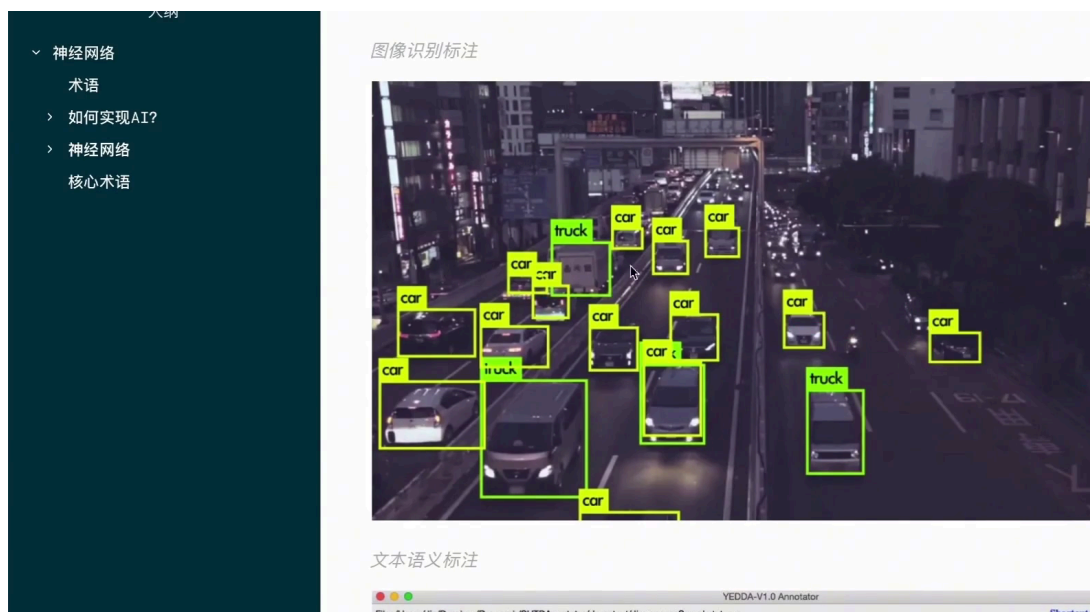
比如识别图片是猫还是狗。给机器一张图，它识别出来可能不对，它也不知道正确答案是什么。这时人去标一下、框一下：这是猫，这是狗。现在有数据标注这种岗位。

给你们看一下。我之前写过一些已经过时的课件，里面有这一块。现在没有按那个课件的方式讲了，我找一下。可以把标注部分截图，放到目前这节课下面；或者不截图也行，这就是数据标注。这些框是人框出来的。

画面说明

画面中监督学习对应“人类给答案”，下方红色方框示意标注区域。其他三类学习方式仍保留，便于比较。

18. 图像标注示例：车辆与目标框



场景 18

完整内容

这些框是人框出来的。人告诉它：这是卡车，这是 car，这是小车。这就是标准答案。

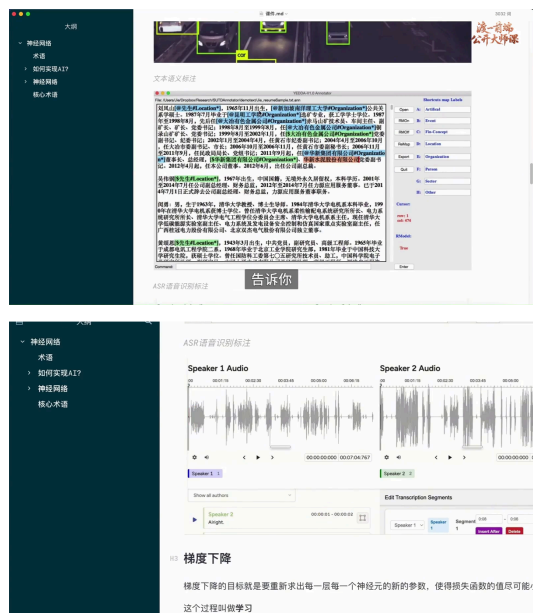
文本标注也一样，告诉它这里是什么，比如这是一个组织，标注一下。AI 自己分出来可能有问题，我们告诉它正确答案是什么，也就是人给正确答案。

声音也可以标注。比如一段混剪音频，可以标出第一秒到第二秒是 Speaker 2，也就是第二个人，说了一个“Alright”。

画面说明

旧课件展示一张街道图，车辆被绿色矩形框圈出，标签中可见 car 和 truck。它是人工提供目标位置与类别的标注例子。

19. 文本与音频的标注示例



场景 19

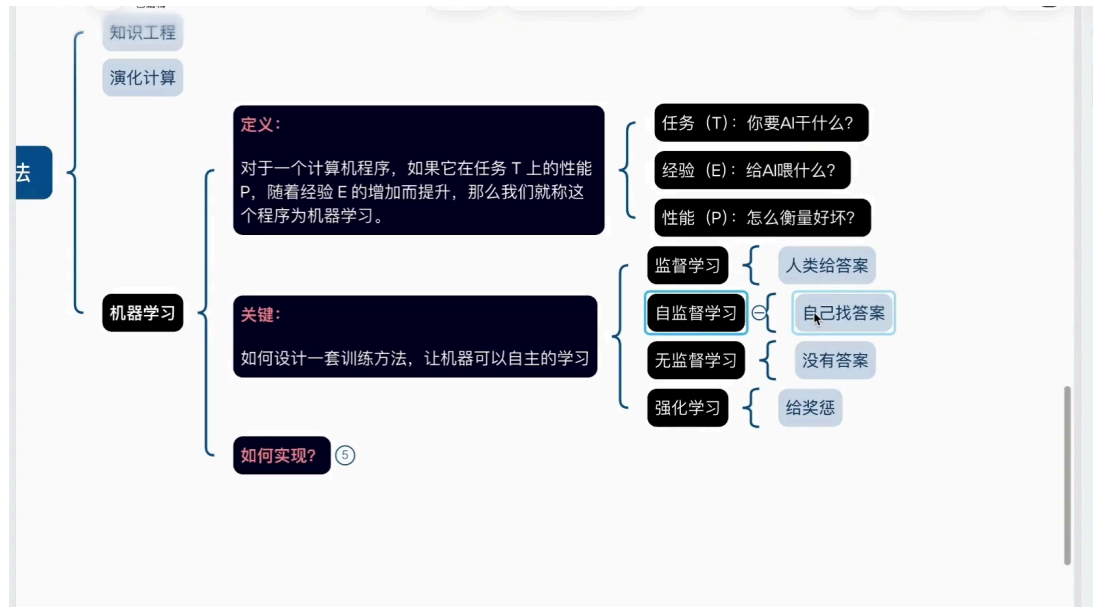
完整内容

第二个人说了一个“Alright”，就是这个意思，这些都能标注。标注的目的，就是告诉 AI 正确答案是什么。注意，这个范式不负责落地。它没有规定必须用这种标注形式、打开某个软件、用一个框去框一下，再告诉 AI。它只说“人要给答案”。怎么给，是你落地的事情，是工程师具体写代码要解决的问题。它讨论的是理论方向。

画面说明

本图由两个实际课件画面上下拼接：上图文本标注界面以不同颜色标记人物、地点、组织等实体；下图 ASR 标注界面包含 Speaker 1 Audio、Speaker 2 Audio 的波形以及一条 Speaker 2 的“Alright.”片段。这里的 Speaker 标签属于示例界面，不是给本课程讲者识别身份。

20. 理论方向与自监督学习的引入



场景 20

完整内容

具体写代码是工程师的事情。我研究的是理论、学术、方向，是人类接下来几十年往哪个方向走；至于落地怎么做，那是另一层问题。

这个世界上有很多只讨论、不亲自做实现的人，但讨论不是没有作用。如果陷入工科思维，觉得什么都一定要落地，就容易忽略这一点。定方向、定思路、讨论办法，其重要性并不比落地低。

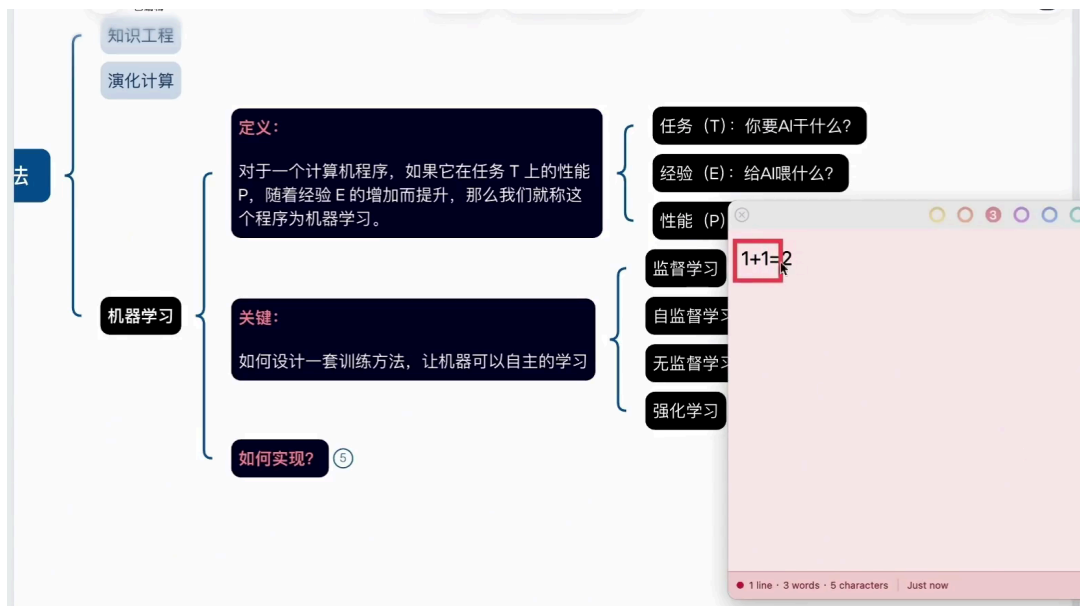
监督学习，就是人给答案。具体展开，它也讲了不少内容，但仍不是工程实现。

还有自监督学习：人不另给答案，你自己找正确答案，有答案，自己找。比如做数学题，我把所有打了一百分的卷子给你，你自己能看到答案。像“ $1+1=2$ ”，喂的时候只喂前面这一部分。

画面说明

画面突出“自监督学习—自己找答案”，并保留“监督学习—人类给答案”等并列节点。讲解强调方向与方法本身有作用，然后转入从现成数据寻找答案。

21. 自监督学习：遮住答案，再自己对答案



场景 21

完整内容

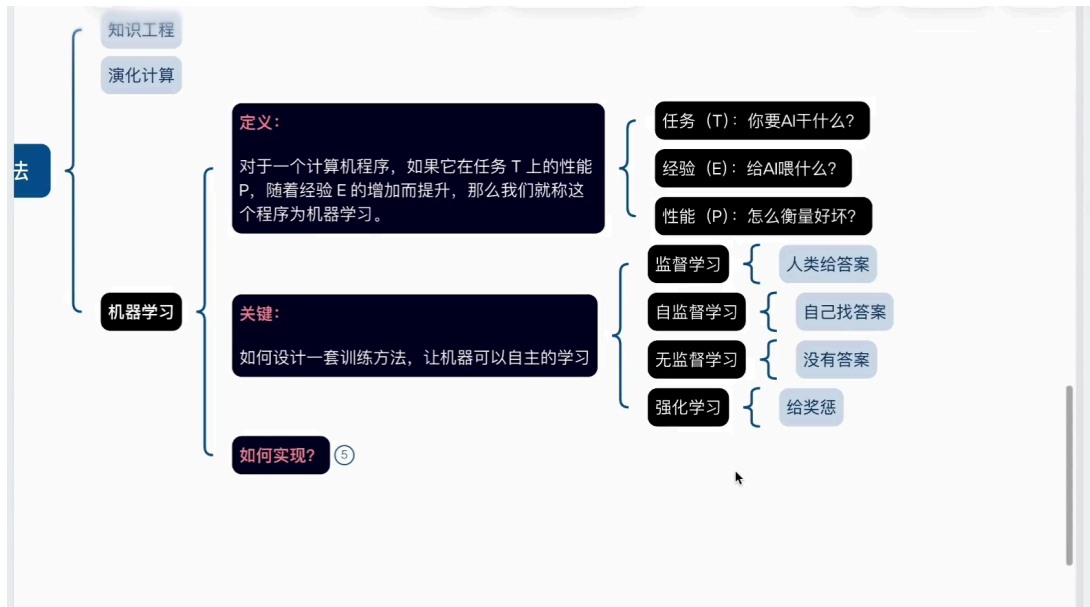
只喂前面这一部分，后面的内容自己猜。怎么知道做得对不对？我们有答案，自己去对，自己产生、取得答案。这就是自监督，不需要人参与，也能自己学习。

监督学习有点像交作业，由老师批改，看你做得对不对。自监督学习则是已经有答案，自己找、自己对。那无监督学习呢？连答案都没有。

画面说明

便笺写出“1+1=2”，其中“1+1”被选中。讲者用已知正确的式子说明只给前半部分，预测后半部分，再利用原有答案核对。

22. 无监督、强化学习与组合使用



场景 22

完整内容

没有答案的情况, 一般用来做分类。比如一个工厂每天产几十万个零件, 就说螺丝吧。大部分是良品, 但少不了十来个或几个残次品。想用摄像头自动看出残次品并剔除, 不需要人工捡。

问题来了: 要识别残次品, 机器学习要训练, 是不是得给它大量残次品? 可哪里有那么多个? 每天都产不了几个。不同残次品又千变万化: 有的螺丝弯了, 有的螺口歪一点, 有的大小不一样, 情况无穷无尽, 但样品又有限。

怎么办? 不去训练所有残次品, 而是训练良品长什么样。凡是跟良品不像的, 就剔除, 不管是哪一种不像。这里讲的是相似度的思路。

再比如古董分类。新挖出一个坑, 里面有瓷器碎片, 属于哪个朝代? 通常得由专家鉴别, 能不能交给 AI? 这些瓷器可能历史上从没出现过, 训练集里没有、没见过, 那怎么办? 还是有办法。

人是怎么做的? 看相似度。纹理跟哪个朝代匹配, 质地跟哪个朝代匹配; 越多特征跟某个朝代匹配, 就判断属于哪个朝代, 进行断代。这里哪里有现成答案? 没有, 靠相似度。大概理解这个意思即可。

还是那句话, 这里没有代码、不负责落地, 只提出思路; 落地是后面的事情。

最后是强化学习, 平时应该听说过, 就是给奖惩。下棋、游戏, 特别适合这个思路。比如围棋下了两百步, 最后输了, 究竟哪一步错了? 我不知道, 我又不懂围棋, 但我知道最后输了, 于是给一次惩罚。

当然, 给机器“一巴掌”没用, 它没有感觉, 肯定要通过数字, 比如传一个负数或某个数值, 让它知道结果不好, 再调整。我说的是奖惩, 不是真打一巴掌。我不知道哪一步错了, 让它自己去调整, 这就叫只给奖惩。

像竞技类游戏、格斗游戏里, 也有很多 AI。我告诉它没赢, 为什么没赢我不知道, 反正输了。或者自己跟自己下, 输的一方也会得到输赢信息。告诉它输了多少、有多差, 它自己调整。

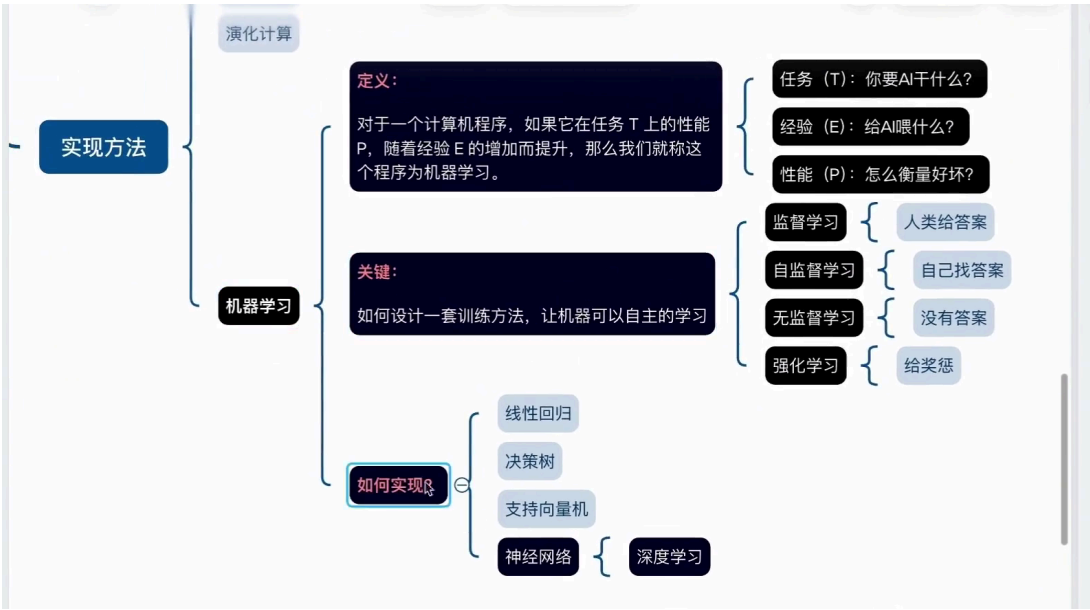
这是强化学习。落地吗? 还是不落地, 只说有这些训练方法、训练手段。真正落地往往结合很多东西。我要做一个 AI, 可能这些全都有, 像 ChatGPT, 不同环节会借助不同的工具与训练范式。这只是一套范式, 告诉你有这些方法。还有半监督学习等, 我就不一一列举了。

它们有道理吗？有。是不是又向前推进一步？是，但还没落地。最后就得问怎么实现机器学习，总归要落地。天天空想有没有用？有用，但最终还是要落地，怎么实现呢？

画面说明

导图将“无监督学习—没有答案”与“强化学习—给奖惩”并列。口述依次用螺丝良品/残次品、瓷器断代、围棋和格斗游戏解释，再强调实际训练可结合多种范式。

23. 机器学习怎样实现：从线性回归到神经网络



场景 23

完整内容

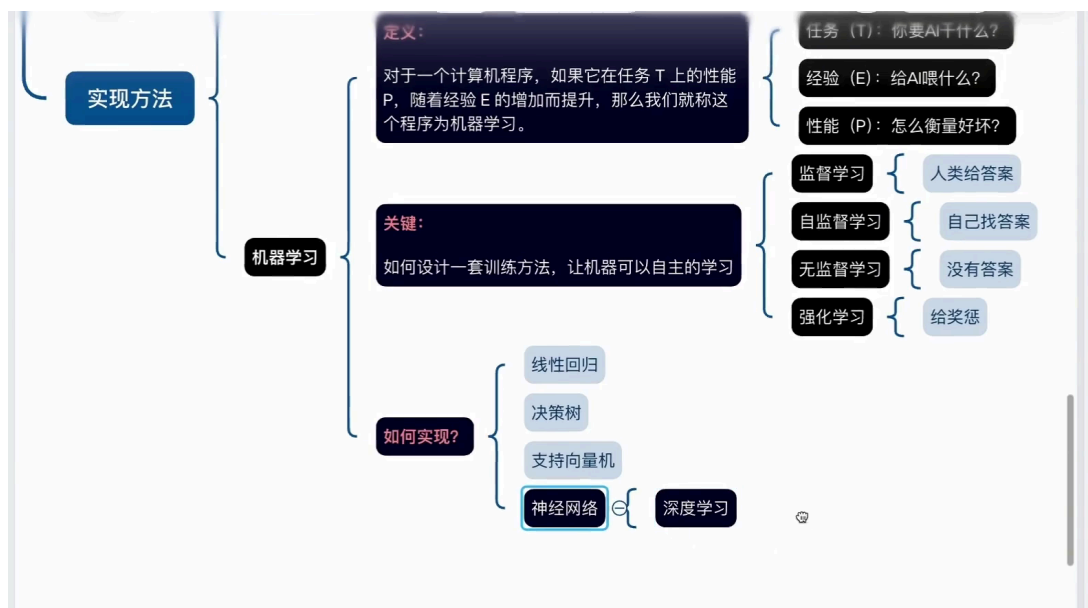
实现手段有很多。网上常刷到“线性回归”，这就是一种实现手段。讲的是是什么？数学原理：空间上有一些散列的点，怎样做出一条线，让它尽量穿过更多点？得到这条线的过程，就是一个学习过程。这里也还没到代码，只是朝落地又推进一大步。

过去用线性回归，还有一些方法曾火过一段时间。这些先不用逐个了解，实现手段太多。现在最流行的是神经网络。你看，图里加黑的都是重要部分。神经网络就是实现机器学习的一种具体手段。

画面说明

“如何实现？”分支列出线性回归、决策树、支持向量机和神经网络；神经网络进一步连接深度学习。这里是实现手段，与上方训练范式是不同分支。

24. 深度学习与强化学习属于不同分类角度



场景 24

完整内容

神经网络是实现机器学习的一种具体手段，但这里落地了吗？也没有，只是设计一套方法，用这套方法可以实现机器学习。这套方法很巧妙，后面会学到。

在神经网络里面，还有一个叫深度学习。深度学习其实就是神经网络。不要把“深度学习”和“强化学习”这两个词搞混，有同学容易混淆。

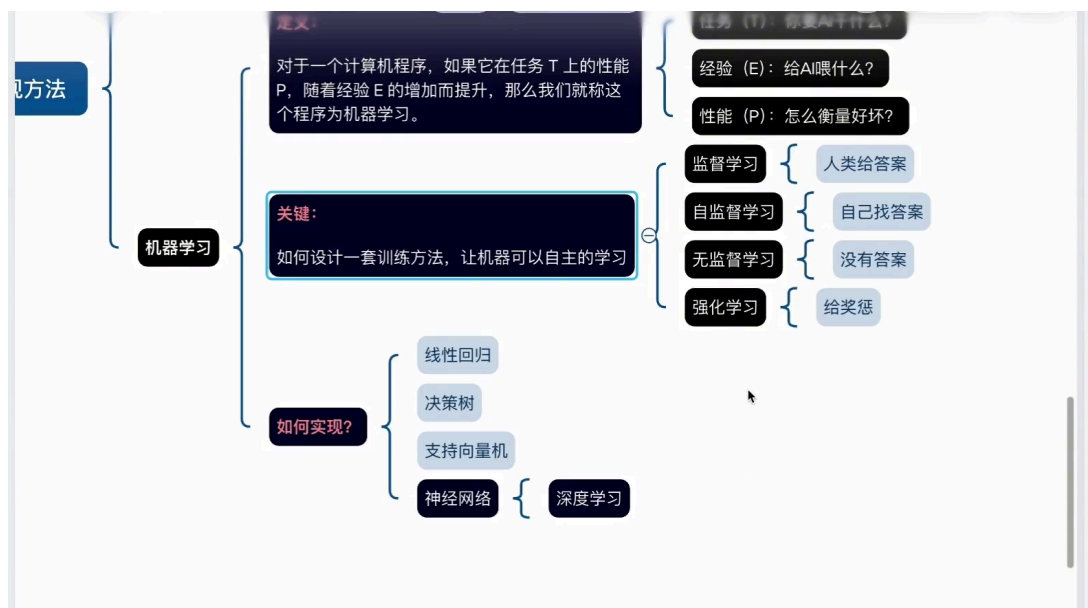
深度学习属于神经网络的一种实现，强化学习则是训练方法。深度学习是实现机器学习的手段，属于神经网络。深度学习一定是神经网络，只是在神经网络基础上，层变得更多、更复杂。后面讲神经网络，就知道了。

最后总结一下。今天聊了很多，实际上没有那么复杂。哲学流派当作历史知识了解就行，没人会专门问某个AI属于哪个哲学流派。

画面说明

神经网络节点连到深度学习；强化学习则位于训练方法分支，对应“给奖惩”。两者不是同一层级的互斥选项。

25. 回顾：方向、问题、训练与实现逐步推进



场景 25

完整内容

没人会专门这样问。要解决的问题，就是各个领域里的问题。各领域有各自的问题，但遇到一个共性问题：没有智能，就解决不了。领域中那些不同的东西研究清楚后，智能由谁提供？

于是讨论智能的实现方法。曾经有很多方法，机器学习是其中一种，也是现在最流行的一种。它想的是：要实现智能，得学习，不学习哪有智能？

但其他路线不一定这样想，它们认为不学习也可以有智能，而且那些方案做出过成绩。机器学习是这些年才占主流，以前主流不是它。

现在这条路强调通过学习获得智能。那么怎么训练？有一些方法，提出这些范式时仍在讨论、不负责落地。怎么落地？落地也需要方法，也需要讨论，慢慢接近，不是一下就能落地，哪有那么容易。

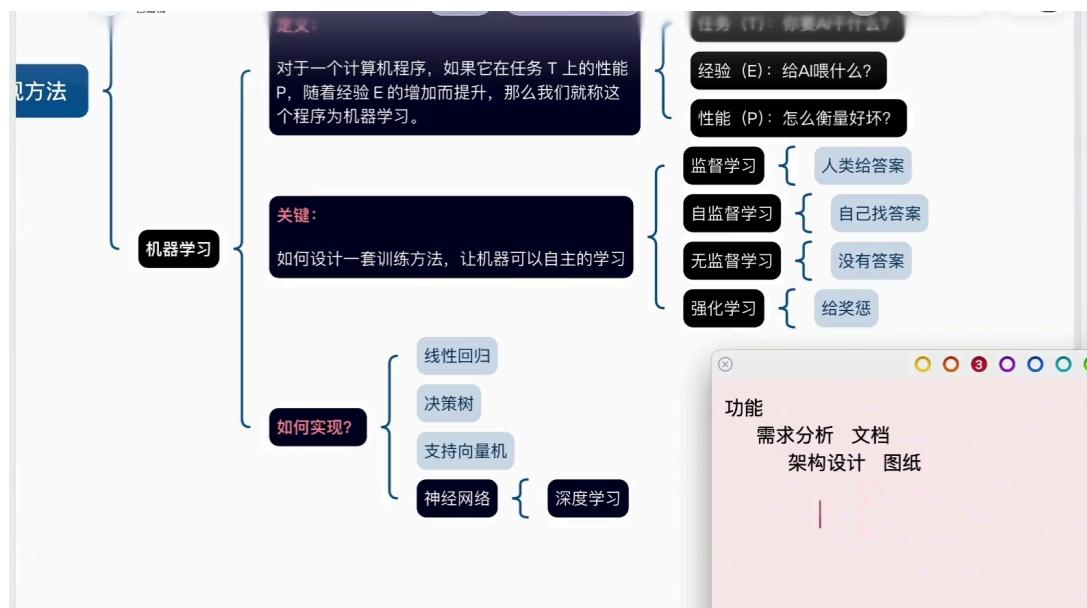
比如做一套系统，得做架构设计。架构设计已经落地了吗？出来的是图纸。神经网络就像一套图纸，告诉你这样可以实现。以前神经网络有比较简易的实现，现在基本是深度学习的实现。

有同学问：“袁老师，不是要讲 Transformer 吗，Transformer 在哪？”它就是深度学习的一种具体实现。Transformer 本身也还不是工程落地，只是后来工程师把它落地了。

画面说明

画面同时保留任务/经验/性能、四种训练范式以及线性回归、决策树、支持向量机、神经网络和深度学习，展示这些术语的层级关系。

26. 从 Transformer 到工程落地，还要继续设计



场景 26

完整内容

这应该是什么关系？给工科生讲这一部分比较恼火。如果给高中生讲，可能会觉得老师这样说有道理，背下来就行。工科生更在意它落不落地，但这里确实还没落地。

工科生说的落地，有点像做一个功能。先做什么？需求分析。产出是文档，哪里有代码？需求分析之后，做架构设计，出来的是图纸。架构设计之后，还要进一步设计。大学里学过，我都快忘了：以前瀑布流程里，Java 那里有类设计、类图设计，出来也还是图。实现要一步一步推进，不是一下就能实现。

现在更多用敏捷开发，没有经过那么长的流程；以前大项目的瀑布式推进，就是这样一层层走。这里也是一样。机器学习是在聊实现，但实现也得逐步推进。

提出实现方案，其中有一套叫神经网络，往前推了一大步；深度学习又推进一步；Transformer 再推进一大步。接下来还需要算法工程师，基于不同行业特点具体设计，还要设计，然后才是落地。是一步一步落地的。

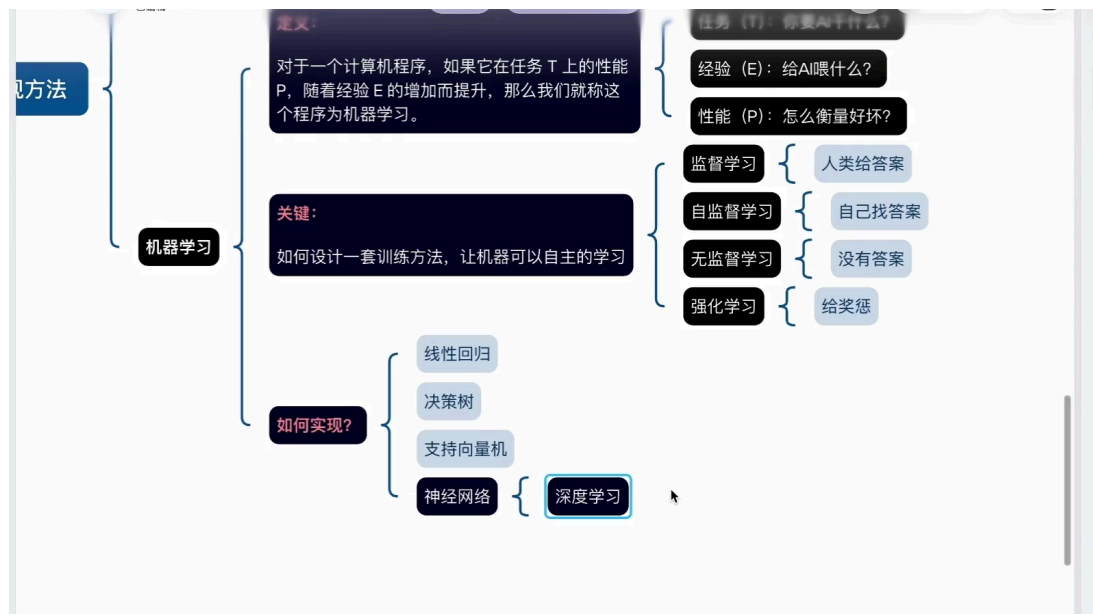
好，这节课讲到这里。能够认识清楚它们的关系就可以，特别是机器学习、神经网络、深度学习之间的关系。对神经网络、深度学习本身还没认识得很清楚也没关系，后面会细化讲，讲了就明白。

但至少清楚机器学习本身与神经网络、深度学习的关系，还包括前面的分类到底是怎样一种分类。

画面说明

便笺列出“功能”“需求分析 文档”“架构设计 图纸”；背景是机器学习、神经网络和深度学习导图。讲者用软件工程逐层设计的过程解释从理论到具体实现仍有步骤。

27. 理解关系，遇到术语就不慌



场景 27

完整内容

把这些认识清楚，以后别人给你蹦词儿、蹦各种名词，你就不会慌了。就是这么一些东西。

画面说明

最后的画面突出“神经网络—深度学习”关系，同时保留机器学习定义、训练范式和其他实现手段。